

Mini Inventory Manager

Object Oriented Programming 2 Project Report

Course	CSC 2210 Object Oriented Programming 2
Domain	Small Retail Shop Inventory Management
Student Name	Tosim Hossain
Student ID	23-55511-3
Section	R
Submission Date	14-09-26

Replace the blank student information before final submission.

1 Introduction

Mini Inventory Manager is a desktop application for a small retail shop. The system gives an administrator a simple way to maintain product records and identify products that are running low. It is built with C# Windows Forms and SQLite.

The project focuses on a clear, explainable flow: a user action on the form triggers an event method, the method sends data to a database helper, SQL updates or reads the database, and the returned data is shown in the DataGridView.

1.1 Objectives

Objective	How the system addresses it
Maintain products	Add, view, update, and delete product records.
Find products quickly	Search by name and filter by category.
Control inventory	Show stock values and a low-stock filter/report.
Demonstrate OOP	Use model classes, inheritance, encapsulation, and database abstraction.

2 Functional Requirements

Requirement	Implemented behavior
Login	The application accepts the demonstration credential admin / 1234.
CRUD	The administrator can add, edit, delete, and view product details.
Filtering	Category, low-stock, and price order controls change the visible products.
Report	A message box lists low-stock products.

3 System Design

The application has two Windows Forms. LoginForm collects the username and password. When the credential is valid, InventoryForm opens. InventoryForm contains the product input controls, action buttons, search and filter controls, and the product grid.

3.1 Database Design

Column	Type	Purpose
Id	INTEGER	Primary key; uniquely identifies a product.
Name	TEXT	Product name.
Category	TEXT	Product category.
Price	REAL	Product selling price.
Stock	INTEGER	Current available quantity.

The database contains one Products table. It is sufficient for this small inventory project because every product has one name, category, price, and stock count.

3.2 Program Flow

User clicks Add button -> AddButton_Click validates the input -> DbHelper.AddProduct executes a parameterised INSERT query -> LoadProducts reads the current products -> DataGridView displays the updated list.

4 Implementation and OOP

Concept	File	Explanation
Encapsulation	Models/Product.cs	Product data is stored in properties instead of loose public fields.
Inheritance	Models/Product.cs	SaleProduct inherits Product and adds DiscountPercent and DiscountedPrice.
Abstraction	Data/DbHelper.cs	Forms call methods such as AddProduct without handling connection details themselves.
Event driven UI	InventoryForm.cs	Button Click events run methods such as AddButton_Click.

Input validation checks that the product name is not blank. The Update and Delete buttons also require the user to select a product first. A confirmation dialog appears before deletion.

5 Database and SQL

DbHelper creates the SQLite database automatically. Its GetProducts method uses a parameterised LIKE query for search text. Parameters are used so user input is passed as data rather than combined directly into SQL.

The low-stock condition is $Stock < 5$. The price order changes between ASC and DESC based on the high-to-low checkbox. The schema.sql file also includes a GROUP BY query that can calculate category-wise inventory value.

6 Testing

Test	Expected Result	Result
Valid login	Inventory form opens.	Pass
Invalid login	Error message appears.	Pass
Add product	New row appears in the grid.	Pass
Blank name	Validation message appears.	Pass
Search/filter	Grid shows matching products.	Pass
Low-stock report	Products under 5 stock are listed.	Pass

7 Limitations and Future Work

This project is intentionally small. It has one demonstration administrator account and one product table. Future versions could add separate user accounts, supplier records, sales invoices, customer ordering, and a richer dashboard.

8 Conclusion

Mini Inventory Manager demonstrates a complete but compact desktop application workflow. It combines Windows Forms controls, event handling, OOP classes, SQLite storage, CRUD operations, filtering, validation, and a stock report in a form that can be explained during a project viva.

Appendix A Screenshots to Attach Before Submission

Insert real screenshots from the running application before submitting this report. Do not use placeholder or edited screenshots.

Required Image	What it should show
Figure 1 Login screen	The Mini Inventory login form with username and password fields.
Figure 2 Inventory dashboard	The DataGridView, Add/Update/Delete controls, and product fields.
Figure 3 Filtering	A search term or category filter changing the displayed products.
Figure 4 Low-stock report	The low-stock filter or low-stock report message.

Save image files as screenshots/login-screen.png and screenshots/inventory-dashboard.png for the README. After adding them to GitHub, update the report by inserting the same images below their captions.